Catch two defects with four test cases

Set up Qase, run the cases manually, then report automated checks.



2 passed

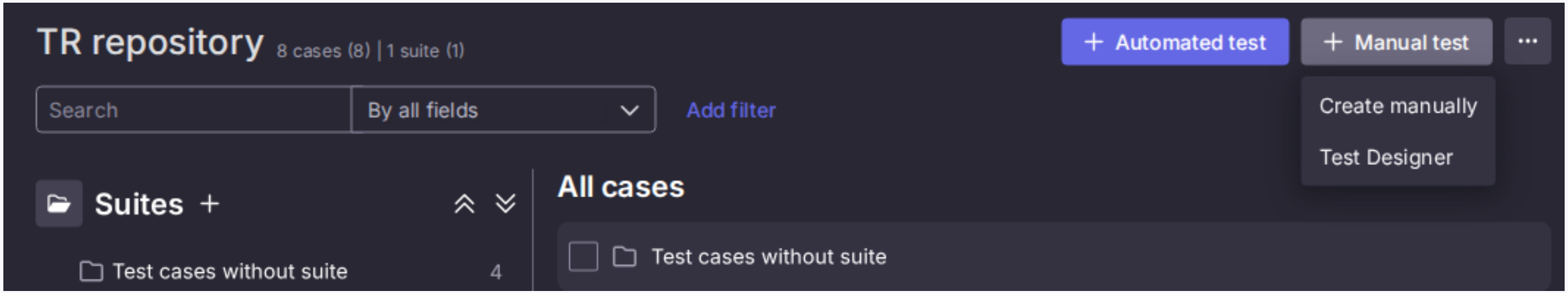
2 failed

Expected in the manual run and each automated suite.

Open lab Open Qase Open GitHub

Start with a Qase project

Prepare access and the place where the cases will live.



1 **Sign in or accept a workspace invitation**

Open Projects, then reuse Triangle, code TR, or create your own project.

2 **Create a suite in Repository**

Use the plus beside Suites. Name it Triangle demo - four cases.

Project > Suite > Case > Run

Connect the two reporters

Configure this once in your Qase workspace and GitHub repository.

Qase Apps > Vitest and Playwright > Activate > Access tokens

Create one token for each app.

GitHub > Settings > Secrets and variables > Actions

Setting name	Where / value
QASE_TESTOPS_PROJECT	Variable / TR
QASE_VITEST_API_TOKEN	Secret / Vitest app token
QASE_PLAYWRIGHT_API_TOKEN	Secret / Playwright app token

The existing workflow installs the reporters and sends both suites to Qase.

Register each case with an expected result

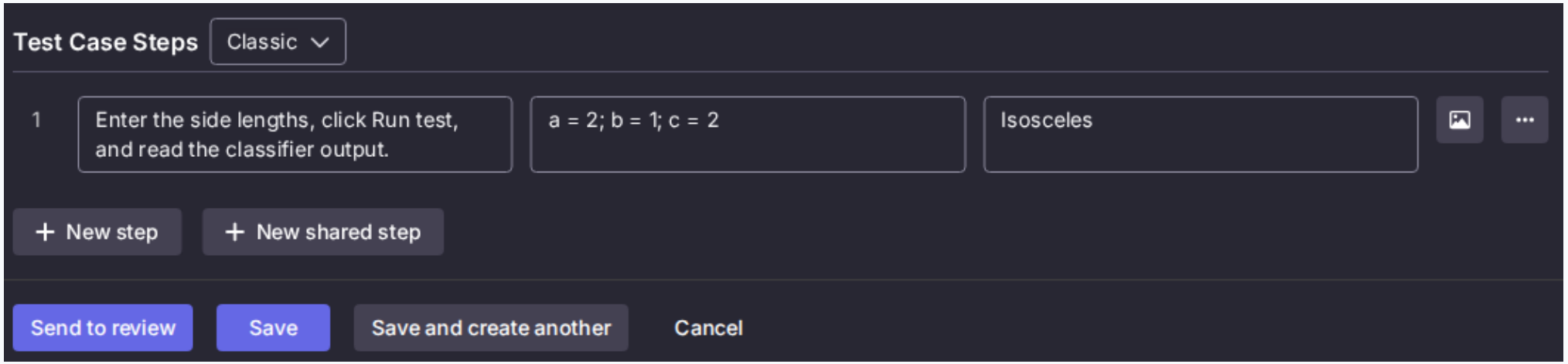
Use the specification when writing the case. Record the faulty output during execution.

1 **Manual test > Create manually**

Choose the suite. Set Automation status to Manual and Type to Functional.

Title **Classifies a = c as Isosceles**

Classic step: action, data, expected result



Repeat with the other three inputs and expected results on the next slide.

Four inputs, four expected results

These are the four cases in the current demonstration.

Case	a, b, c	Expected	Actual output	Record
01 a and c match	2, 1, 2	Isosceles	Scalene	**Failed**
02 200 is allowed	1, 200, 200	Isosceles	Value is out of range	**Failed**
03 Equilateral	3, 3, 3	Equilateral	Equilateral	**Passed**
04 201 rejected	201, 1, 1	Value is out of range	Value is out of range	**Passed**

Whole numbers 1 through 200 are allowed, including 200.

Run, compare, record in Qase

The expected result belongs to the case. The actual result comes from Triangle Lab.

1. **Select the four manual cases > Run**

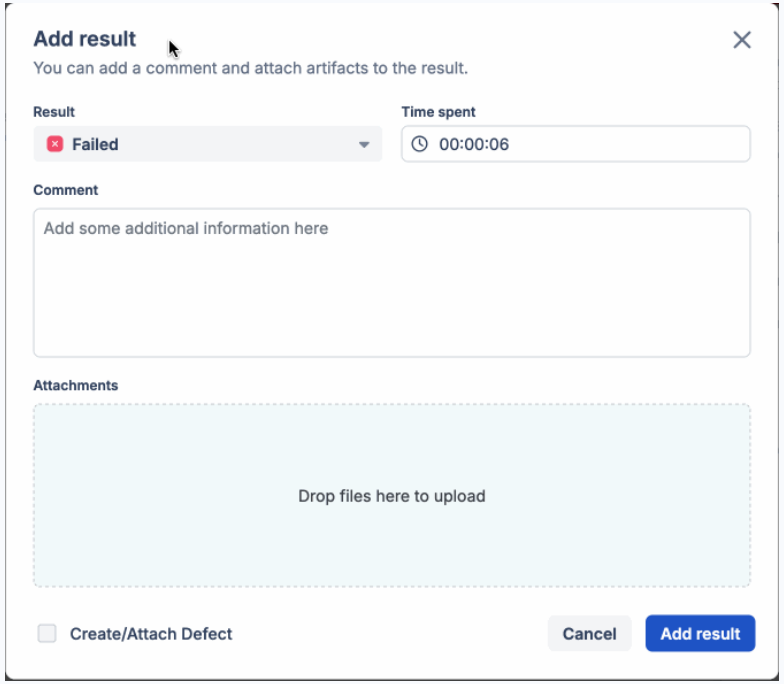
   Name the run Triangle demo - Manual. Open its wizard.

2. **Enter sides in the lab > Run test**

   Read the faulty classifier's output, then compare it with the case.

3. **Save Passed or Failed in Qase**

   For a failure, add a comment and attach the lab screenshot.



Add result dialog. Official Qase help example.

Example Comment

a=2, b=1, c=2. Expected: Isosceles. Actual: Scalene.

Start the existing GitHub workflow

In your repository or fork, run both suites once and inspect Qase.

GitHub Actions
Runs the checked-out tests

>

Vitest + Playwright
Four checks in each suite

>

Qase project TR
Two reports with results

1. **Actions > Performative defect detection**
   Click Run workflow, choose main, then Run workflow again.
2. **Check logs, then open Qase Test Runs**
   Confirm reporting is enabled and that both suites uploaded four results.

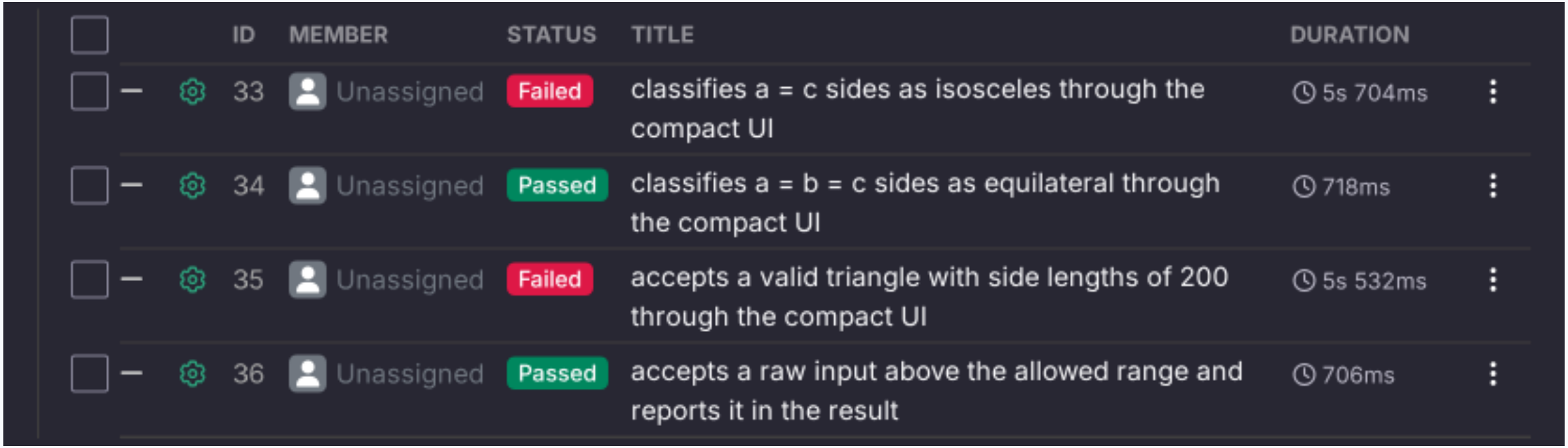
Open workflow

Two assertion failures per suite should make the job red.

Inspect the uploaded Qase results

Open the newest Direct Effect and Browser UI runs. Each should have four checks.

Example: Browser UI, run 16



ID	MEMBER	STATUS	TITLE	DURATION
33	Unassigned	Failed	classifies a = c sides as isosceles through the compact UI	5s 704ms
34	Unassigned	Passed	classifies a = b = c sides as equilateral through the compact UI	718ms
35	Unassigned	Failed	accepts a valid triangle with side lengths of 200 through the compact UI	5s 532ms
36	Unassigned	Passed	accepts a raw input above the allowed range and reports it in the result	706ms

2 passed + 2 failed in each suite

Open a Failed case for assertion details.

Follow along on your own machine

Use Git, Node.js 22.18 and pnpm 11.9.0 to match the repository workflow.

1 Install from the repository

```
git clone \
https://github.com/tiara-stack/triangle.git
cd triangle
pnpm install --frozen-lockfile
```

Install Chromium

```
node node_modules/@playwright/test/cli.js \
install chromium
```

2 Copy .env.local.example to .env.local

```
QASE_TESTOPS_PROJECT=TR

QASE_VITEST_API_TOKEN=<Vitest token>

QASE_PLAYWRIGHT_API_TOKEN=<Playwright token>
```

Replace the placeholders with your own tokens.

```
3 pnpm test:performative:qase
```

Show the evidence, then the conclusion

Use the passing cases to establish a baseline and the failures to demonstrate detection.

1. **Before the session**
   Qase access, four manual cases, GitHub settings and one rehearsal.

2. **During the demonstration**
   Start automation. Run manual cases 03, 01, 02, 04. Complete the manual run.

3. **Show the reports**
   Each run has 2 passed + 2 failed. Open a failed assertion and explain the mismatch.

The classifier is faulty. These tests catch it.

Triangle Lab

Qase repository

GitHub workflow